

共享室内场景三维高斯重建完整实验报告

Jingyi Guo PB22011958 Siqi Han PB22010386 Yiru Wang PB22081510

2026 年 6 月

1 摘要

本报告记录 Assignment 10 Stage 2 共享室内场景重建实验的完整过程。任务要求在共享室内测试场景上完成 3D Gaussian Splatting 重建，输出可渲染的新视角图像，计算 PSNR 和 SSIM，给出几何精度评估，完成消融实验或对比实验，并分析失败区域和改进方向。实验覆盖 Sequence 01 到 Sequence 05 五个共享序列。完整流程包括图像筛选与分区、COLMAP 相机位姿估计、稀疏点云初始化、基于 gsplat 的受控 3DGS 训练、增密和 finetune、SuperSplat 兼容 PLY 导出、图像质量评估、点云几何评估、局部测量评估、Mip Splatting、2D Gaussian Splatting 和 Depth RegularizedGS 对比。

本实验的主结果来自自写训练脚本 `scripts/train_stage2_gsplat.py`。该脚本直接读取 COLMAP 输出的 `cameras.txt`、`images.txt` 和 `points3D.txt`，使用 COLMAP 稀疏点初始化三维高斯，调用 `gsplat.rasterization` 完成 CUDA 可微光栅化，并实现了损失函数、梯度驱动增密、训练日志、渲染保存、指标计算和 checkpoint 保存。最终五个序列平均 PSNR 分别为 26.36 dB、30.15 dB、26.33 dB、28.36 dB 和 26.39 dB。Sequence 04 的受控 3DGS finetune 达到 28.36 dB，Mip Splatting 达到 27.77 dB，2DGS 达到 27.18 dB。Depth RegularizedGS 在本环境中完成训练和评估，但平均 PSNR 为 15.92 dB，因此作为失败对照保留。

视觉观察显示，黑色线条、门框、线缆、文字和地毯色块恢复较稳定。白色柜面、白色墙面和反光区域恢复较差。原因在于高对比结构能够提供稳定的 SIFT 匹配和清晰的图像梯度，弱纹理区域缺少可靠特征，反光区域还具有随视角变化的高光位置。该观察与 SfM 特征提取机制和 3DGS 静态颜色优化机制一致。后续改进应重点采用反光 mask、稠密单目深度、线段监督、平面约束、Mip Splatting 长训练和空间分区拼接。

2 成员与分工

表 1: 成员信息与分工

成员	学号	分工
Jingyi Guo	PB22011958	数据准备, COLMAP 重建, 受控 3DGS 训练, 脚本整合, 报告整合, GitHub 仓库组织
Siqi Han	PB22010386	进阶方法调研, Mip Splatting 与 2DGS 对照, 反光区域诊断, 后续优化规划
Yiru Wang	PB22081510	几何精度评估, 局部测量验证, 结果表核查, 答辩材料整理

3 任务定义

3.1 作业要求

Stage 2 的目标是在共享室内场景上完成高质量 3DGS 重建。作业评分包括四个部分。重建质量要求共享测试场景能够训练完成并渲染新视角, PSNR 达到 20 dB、25 dB 和 28 dB 对应不同档位。精度评估要求有可运行脚本、至少五组测量对比数据, 并尽量达到 20 cm 和 10 cm 精度。问题分析与创新改进要求明确记录失败区域, 解释原因, 使用超越基础流程的改进方案, 并用数据证明改善。技术文档要求包含 Introduction、Method、Experiments 和 Conclusion, 并包含流程图、定量结果、对比分析和失败案例讨论。

本实验按照这些要求组织。图像质量用 PSNR 和 SSIM 评价。几何精度用重建高斯中心点云与共享参考点云之间的相似变换配准误差评价, 并补充五个局部区域的测量误差。创新改进包括数据侧筛选、分区训练、COLMAP robust 参数、受控 densification、低学习率 finetune、Mip Splatting、2DGS 和 Depth RegularizedGS。

3.2 数据集

数据来自作业提供的共享室内测试场景, 解压后位于 `data`。实验处理了五个序列, 分别为 Sequence 01 到 Sequence 05。每个序列包含多张室内图像以及对应参考几何数据。室内场景包含柜体、墙面、地面、门框、地毯、线缆、玻璃或反光表面等。与 Stage 1 小物体场景相比, Stage 2 的难点包括大尺度相机轨迹、弱纹理平面、重复纹理、反光材质、视角遮挡和更高计算量。

实验中的工作目录为 `work\Sequence_xx\variant_name`。每个 variant 包含输入图像、COLMAP 输出、训练输出、渲染结果、指标表和几何评估结果。最终使用的 variant 列于表 2。

表 2: 最终采用的序列变体

序列	最终变体	选择原因
Sequence 01	enhanced_long_finetune	完整注册后进行较长 finetune
Sequence 02	partition_mid_finetune	中段分区注册稳定, 最终 PSNR 最高
Sequence 03	partition_best_finetune	分区训练显著提升弱基线
Sequence 04	partition_best_finetune	渲染质量和几何质量较均衡, 也是进阶方法对比目标
Sequence 05	full_finetune	使用完整大序列并进行 finetune

3.3 环境

实验在 Windows 环境中运行。Python 解释器为

```
C:\Users\gjy20\.conda\envs\exp3\python.exe
```

实际环境记录在 docs/REPRODUCE.md 中。主要运行组件为 PyTorch 2.5.1+cu121、CUDA runtime 12.1、NVIDIA GeForce RTX 4060 Laptop GPU、gsplat 1.5.3、NumPy 2.4.0、Pillow 12.0.0 和 scikit-image 0.26.0。CUDA 扩展编译使用本地 CUDA 12.1 工具链和 Visual Studio developer command。COLMAP 调用来自 Stage 1 已安装工具。

4 方法总览

完整流程见图 1。输入图像经过筛选与分区后进入 COLMAP。COLMAP 输出相机内参、相机外参和稀疏点云。受控 3DGS 训练脚本读取这些结果, 将稀疏点转换为初始三维高斯, 并通过可微渲染优化位置、尺度、旋转、颜色、不透明度和背景颜色。训练期间记录 loss、PSNR、高斯数量并执行梯度增密。训练结束后保存渲染图、真实图与渲染图对比、指标表、checkpoint 和 SuperSplat PLY。最后使用图像指标和几何指标评估结果。

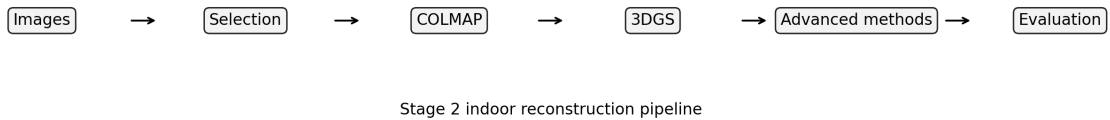


图 1: Stage 2 共享室内场景重建流程

本项目的实现由公开工具、论文代码和自写脚本共同组成。COLMAP、Mip Splatting、2DGS 和 Depth RegularizedGS 的核心算法来自公开工具和论文实现。受控 3DGS 主流程中的数据读取、图像缩放、相机矩阵构造、高斯初始化、损失函数、增密策略、训练日志、指标计算、PLY 导出和几何评估由本项目脚本实现。CUDA 可微渲染核心调用 gsplat 的 `rasterization` 函数。该函数接收高斯中心、四元数、尺度、不透明度、颜色、相机 view matrix、内参矩阵和图像尺寸, 输出渲染图像。

5 数据筛选与分区

5.1 筛选动机

室内共享序列的图像质量不均匀。相邻帧可能信息重复，模糊帧会降低特征匹配质量，过暗或过曝帧会降低图像监督质量。直接把全部图像输入 COLMAP 和 3DGS 会增加计算量，并可能让低质量片段影响整体优化。实验中使用两类数据处理方法。第一类为质量筛选，按照清晰度、亮度和对比度选择图像。第二类为分区，选择连续的稳定相机片段，使局部场景和相机轨迹更容易拟合。

5.2 质量筛选代码

质量筛选脚本为 `scripts/prepare_stage2_sequence.py`。核心思想是用 Laplacian 方差估计清晰度，用平均亮度和灰度标准差估计曝光与对比度，并在每个时间分段中选出分数最高的帧。关键代码如下。

Listing 1: 图像质量评分和 sharp 模式筛选

```

1 def image_quality(path: Path) -> dict:
2     image = cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)
3     if image is None:
4         raise ValueError(f"Cannot read image: {path}")
5     laplacian_var = float(cv2.Laplacian(image, cv2.CV_64F).var())
6     mean = float(image.mean())
7     std = float(image.std())
8     exposure_penalty = abs(mean - 128.0) * 0.25 + max(0.0, 35.0 - std) * 2.0
9     score = laplacian_var - exposure_penalty
10    return {
11        "laplacian_var": laplacian_var,
12        "mean": mean,
13        "std": std,
14        "score": score,
15    }
16
17 def choose_sharp(files: list[Path], target_count: int, metrics: dict[Path, dict]) -> list[Path]:
18     if len(files) <= target_count:
19         return files
20     bins = np.array_split(np.arange(len(files)), target_count)
21     chosen = []
22     for bin_indices in bins:
23         candidates = [files[int(index)] for index in bin_indices]
24         best = max(candidates, key=lambda item: metrics[item]["score"])
25         chosen.append(best)
26     return chosen

```

该实现先把时间轴划分为若干 bin，再在每个 bin 中选择最清晰且曝光较稳定的帧，避免所有高分图像集中到某一小段。这样可以同时保留视角覆盖和图像质量。

5.3 分区代码

分区脚本为 `scripts/prepare_stage2_partition.py`。它按一段连续的一基索引区间提取图像，并保留 stride 参数。关键代码如下。

Listing 2: 连续片段分区

```

1 selected = files[args.start_index - 1 : args.end_index : args.stride]
2 input_dir = args.work_dir / "input"
3 if input_dir.exists() and args.overwrite:
4     shutil.rmtree(input_dir)
5 input_dir.mkdir(parents=True, exist_ok=True)
6
7 rows = []
8 for new_index, src in enumerate(selected, start=1):
9     dst_name = f"frame_{new_index:05d}.jpg"
10    dst = input_dir / dst_name
11    copy_image(src, dst, args.max_width)
12    rows.append(
13        {
14            "new_name": dst_name,
15            "source_name": src.name,
16            "source_index": files.index(src) + 1,
17            **image_quality(src),
18        }
19    )

```

分区在 Sequence 03 和 Sequence 04 上带来明显提升。原因是大场景中单个模型需要同时解释多个空间区域和大量视角变化，局部连续片段更容易获得稳定 COLMAP 注册和稳定图像监督。

6 COLMAP 和多视图几何

6.1 相机投影模型

多视图重建首先需要估计每张图像的相机位姿。三维点 $\mathbf{X}_j$ 到二维观测 $\mathbf{x}_{ij}$ 的投影关系可以写为

$$\tilde{\mathbf{x}}_{ij} \sim K_i \begin{bmatrix} R_i & \mathbf{t}_i \end{bmatrix} \tilde{\mathbf{X}}_j.$$

其中 K_i 为相机内参，包含焦距和主点。 R_i 与 $\mathbf{t}_i$ 为相机外参，描述相机朝向和位置。3DGS 需要这些参数，因为每次训练都要从某一个相机位置把三维高斯投影成二维图像，再和真实图像比较。

6.2 SfM 原理

Structure from Motion 的目标是在场景静止且相机运动的条件下，同时恢复相机运动和三维结构。COLMAP 是通用 SfM 与 MVS 工具，其增量式 SfM 流程来自 Schönberger 和 Frahm 的工作 [3, 2]。实际步骤包括 SIFT 特征提取、特征匹配、RANSAC 几何验证、选择初始图像对、三角化、图像注册和 Bundle Adjustment。

SIFT 由 Lowe 提出 [4]。它在尺度空间中寻找稳定关键点，并用局部梯度方向构造描述子。不同图像中的描述子相似时，可能对应同一个物理点。匹配会受到重复纹理和弱纹理影响，因此 COLMAP 使用 RANSAC 过滤外点。RANSAC 由 Fischler 和 Bolles 提出 [5]，通过多次随机采样估计模型，并选择被最多内点支持的模型。

如果同一个物理点在两张或多张图像中出现，可以通过三角化估计三维位置。理想情况下，两台相机中心到对应像素的射线在三维空间相交。真实数据中存在噪声，算法求解使重投影误差最小的三维点。

Bundle Adjustment 对所有相机和三维点进行联合优化 [6]。目标函数为

$$\min_{\{R_i, \mathbf{t}_i\}, \{\mathbf{X}_j\}} \sum_{i,j} \|\pi(K_i(R_i \mathbf{X}_j + \mathbf{t}_i)) - \mathbf{x}_{ij}\|_2^2.$$

这里 π 是透视除法后的像素投影函数。该优化使 COLMAP 输出的相机位姿和稀疏点云在同一坐标系内保持一致。

6.3 COLMAP 参数

COLMAP 脚本为 `scripts/run_stage2_colmap.ps1`。该脚本执行 feature extractor、matcher、mapper、image undistorter 和 model converter。robust 配置的关键参数如下。

Listing 3: COLMAP robust 配置中的特征提取参数

```
1 "feature_extractor",
2 "--ImageReader.single_camera", "1",
3 "--ImageReader.camera_model", "SIMPLE_RADIAL",
4 "--FeatureExtraction.use_gpu", "1",
5 "--SiftExtraction.max_num_features", "16384",
6 "--SiftExtraction.peak_threshold", "0.003",
7 "--SiftExtraction.edge_threshold", "20",
8 "--SiftExtraction.estimate_affine_shape", "1",
9 "--SiftExtraction.domain_size_pooling", "1"
```

这些参数提高了 SIFT 特征数量，降低了峰值阈值，并开启 affine shape 和 domain size pooling。这样做的目标是在弱纹理和视角变化较大的室内图像中保留更多可匹配特征。

Listing 4: COLMAP robust 配置中的匹配和 mapper 参数

```
1 "exhaustive_matcher",
2 "--FeatureMatching.use_gpu", "1",
3 "--FeatureMatching.guided_matching", "1",
4 "--TwoViewGeometry.min_num_inliers", "10",
5 "--TwoViewGeometry.min_inlier_ratio", "0.15"
6
7 "mapper",
8 "--Mapper.ba_global_function_tolerance", "0.000001",
9 "--Mapper.init_min_num_inliers", "30",
10 "--Mapper.init_min_tri_angle", "4",
11 "--Mapper.abs_pose_min_num_inliers", "12",
12 "--Mapper.abs_pose_min_inlier_ratio", "0.10",
13 "--Mapper.ba_refine_principal_point", "1"
```

匹配阶段使用 guided matching，mapper 阶段降低初始化和绝对位姿估计的内点要求。该设置提高了 Sequence 01、03、04、05 的注册完整性。Sequence 02 最终变体的输入图像为 80 张，成功注册 49 张，说明该序列仍存在较强的匹配困难。

7 受控 3DGS 实现

7.1 三维高斯表示

3D Gaussian Splatting 由 Kerbl、Kopanas、Leimkühler 和 Drettakis 于 2023 年提出 [1]。该方法用一组三维高斯表示场景。第 i 个高斯包含中心 μ_i 、尺度 s_i 、旋转 R_i 、颜色 $\mathbf{c}_i$ 和不透明度 α_i 。三维高斯函数为

$$G_i(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_i)^T \Sigma_i^{-1}(\mathbf{x} - \mu_i)\right).$$

协方差用尺度和旋转分解

$$\Sigma_i = R_i S_i S_i^T R_i^T, \quad S_i = \text{diag}(s_{i,0}, s_{i,1}, s_{i,2}).$$

训练时，中心、尺度、旋转、颜色和不透明度都作为可学习参数。高斯投影到相机平面后形成二维椭圆 footprint，并通过 alpha 合成得到像素颜色

$$\mathbf{C}(u, v) = \sum_{i=1}^N T_i(u, v) \alpha_i(u, v) \mathbf{c}_i, \quad T_i(u, v) = \prod_{k < i} (1 - \alpha_k(u, v)).$$

该公式表达前后遮挡和透明度累积。原始 3DGS 论文的关键贡献包括从 SfM 稀疏点初始化、优化各向异性协方差、交替优化和密度控制，以及快速 visibility aware splatting [1]。

7.2 训练脚本输入

训练脚本 `scripts/train_stage2_gsplat.py` 的命令行参数如下。默认迭代次数为 7000，最终大部分实验使用更高迭代次数和 finetune。

Listing 5: 受控 3DGS 训练脚本参数

```

1 parser.add_argument("--scene-dir", type=Path, required=True)
2 parser.add_argument("--output-dir", type=Path, required=True)
3 parser.add_argument("--figures-dir", type=Path, required=True)
4 parser.add_argument("--iterations", type=int, default=7000)
5 parser.add_argument("--max-side", type=int, default=768)
6 parser.add_argument("--seed", type=int, default=22011958)
7 parser.add_argument("--eval-every", type=int, default=500)
8 parser.add_argument("--save-render-count", type=int, default=12)
9 parser.add_argument("--densify-until", type=int, default=3500)
10 parser.add_argument("--densify-every", type=int, default=500)
11 parser.add_argument("--densify-fraction", type=float, default=0.08)
12 parser.add_argument("--densify-max-new", type=int, default=1600)
13 parser.add_argument("--lambda-ssim", type=float, default=0.0)
14 parser.add_argument("--lambda-edge", type=float, default=0.0)
15 parser.add_argument("--lambda-opacity", type=float, default=0.0)
16 parser.add_argument("--resume-checkpoint", type=Path, default=None)
17 parser.add_argument("--lr-scale", type=float, default=1.0)

```

这些参数对应实际调参入口。我们主要调节 iterations、max side、densify until、densify interval、densify fraction、densify max new、SSIM 权重、edge 权重、opacity 正则权重、resume checkpoint 和 finetune 学习率缩放。

7.3 COLMAP 输出读取

训练脚本读取 `sparse_text` 下的三类 COLMAP 文本文件。相机文件提供内参，图像文件提供四元数和平移，三维点文件提供稀疏点坐标和颜色。关键代码如下。

Listing 6: 读取 COLMAP 相机、图像位姿和三维点

```

1 def read_cameras(path: Path) -> dict[int, Camera]:
2     cameras: dict[int, Camera] = {}
3     for line in path.read_text(encoding="utf-8").splitlines():
4         if not line or line.startswith("#"):
5             continue
6         parts = line.split()
7         camera_id = int(parts[0])
8         model = parts[1]
9         width = int(parts[2])
10        height = int(parts[3])
11        params = [float(x) for x in parts[4:]]
12        cameras[camera_id] = Camera(camera_id, model, width, height, params)
13    return cameras
14
15 def read_images(path: Path) -> list[ImagePose]:
16     lines = [line for line in path.read_text(encoding="utf-8").splitlines()
17              if line and not line.startswith("#")]
18     poses: list[ImagePose] = []
19     for i in range(0, len(lines), 2):
20         parts = lines[i].split()
21         image_id = int(parts[0])
22         qvec = np.array([float(x) for x in parts[1:5]], dtype=np.float64)
23         tvec = np.array([float(x) for x in parts[5:8]], dtype=np.float64)
24         camera_id = int(parts[8])
25         name = parts[9]
26         poses.append(ImagePose(image_id, qvec, tvec, camera_id, name))
27    return sorted(poses, key=lambda item: item.name)
28
29 def read_points(path: Path) -> tuple[np.ndarray, np.ndarray]:
30     points = []
31     colors = []
32     for line in path.read_text(encoding="utf-8").splitlines():
33         if not line or line.startswith("#"):
34             continue
35         parts = line.split()
36         points.append([float(parts[1]), float(parts[2]), float(parts[3])])
37         colors.append([int(parts[4]), int(parts[5]), int(parts[6])])
38    return np.asarray(points, dtype=np.float32), np.asarray(colors, dtype=np.float32) / 255.0

```

7.4 相机矩阵和图像缩放

训练时图像按最大边缩放到 `--max-side`。相机内参必须同步缩放，否则三维点会投影到错误像素位置。脚本中的相机内参缩放代码如下。

Listing 7: 内参缩放

```

1 def camera_intrinsics(camera: Camera, scaled_width: int, scaled_height: int) -> np.ndarray:
2     scale_x = scaled_width / camera.width
3     scale_y = scaled_height / camera.height

```



```

4     if camera.model == "PINHOLE":
5         fx, fy, cx, cy = camera.params[:4]
6     elif camera.model in {"SIMPLE_PINHOLE", "SIMPLE_RADIAL"}:
7         f, cx, cy = camera.params[:3]
8         fx = f
9         fy = f
10    else:
11        fx, fy, cx, cy = camera.params[:4]
12    return np.array(
13        [[fx * scale_x, 0.0, cx * scale_x],
14         [0.0, fy * scale_y, cy * scale_y],
15         [0.0, 0.0, 1.0]],
16        dtype=np.float32,
17    )

```

COLMAP 的旋转用四元数保存,脚本将四元数转换为旋转矩阵,并和位移拼成 4×4 view matrix。每张图像对应一个 view matrix 和一个内参矩阵。

7.5 初始化

3DGS 原论文从 SfM 稀疏点初始化三维高斯 [1]。本实验使用同样思路。稀疏点位置作为高斯中心,稀疏点颜色作为初始颜色。尺度由最近邻距离估计。不透明度初值为 0.12,四元数初值为单位旋转,背景颜色由图像角落平均颜色估计。代码如下。

Listing 8: 高斯初始化

```

1  means = torch.nn.Parameter(torch.from_numpy(points).to(device))
2  log_scales = torch.nn.Parameter(
3      torch.log(torch.from_numpy(scale_values).to(device).view(-1, 1).repeat(1, 3))
4  )
5  quats_init = torch.zeros((points.shape[0], 4), dtype=torch.float32, device=device)
6  quats_init[:, 0] = 1.0
7  quats = torch.nn.Parameter(quats_init)
8  opacity_logits = torch.nn.Parameter(
9      logit(torch.full((points.shape[0], 1), 0.12, dtype=torch.float32, device=device))
10 )
11 color_logits = torch.nn.Parameter(logit(torch.from_numpy(point_colors).to(device)))
12 background_logits = torch.nn.Parameter(logit(corner_pixels.mean(dim=0).to(device)))

```

最近邻尺度估计避免所有高斯一开始过大或过小。脚本对最近邻距离做 5 分位和 95 分位裁剪,再乘以 0.85。该处理降低了离群稀疏点对尺度初始化的影响。

7.6 优化器

不同参数的量纲和敏感性不同,因此训练使用 Adam 的多个参数组。中心学习率按场景尺度缩放,尺度、旋转、不透明度、颜色和背景使用不同学习率。

Listing 9: Adam 参数组学习率

```

1  return torch.optim.Adam(
2      [
3          {"params": [means], "lr": 0.00025 * scene_extent * lr_scale},
4          {"params": [log_scales], "lr": 0.006 * lr_scale},
5          {"params": [quats], "lr": 0.001 * lr_scale},

```

```

6         {"params": [opacity_logits], "lr": 0.05 * lr_scale},
7         {"params": [color_logits], "lr": 0.015 * lr_scale},
8         {"params": [background_logits], "lr": 0.01 * lr_scale},
9     ],
10     eps=1e-15,
11 )

```

finetune 阶段使用 `--resume-checkpoint` 读取已有模型，并用 `--lr-scale` 降低所有参数组学习率。Sequence 04 的最终 finetune 使用 10000 次额外迭代，学习率缩放为 0.35。

7.7 可微渲染和损失函数

可微渲染核心调用 `gsplat` 文档中的 `rasterization` 接口 [8]。本项目传入高斯参数和相机参数，输出当前相机视角的渲染图。训练损失为

$$\mathcal{L} = 0.8\mathcal{L}_1 + 0.2\mathcal{L}_{mse} + \lambda_{ssim}(1 - \text{SSIM}) + \lambda_{edge}\mathcal{L}_{edge} + \lambda_{opacity}\bar{\alpha}.$$

其中

$$\mathcal{L}_1 = \frac{1}{HW} \sum_{u,v} \|\hat{\mathbf{I}}(u,v) - \mathbf{I}(u,v)\|_1, \quad \mathcal{L}_{mse} = \frac{1}{HW} \sum_{u,v} \|\hat{\mathbf{I}}(u,v) - \mathbf{I}(u,v)\|_2^2.$$

边缘损失使用 Sobel 梯度

$$\mathcal{L}_{edge} = \|\nabla_x \hat{\mathbf{I}} - \nabla_x \mathbf{I}\|_1 + \|\nabla_y \hat{\mathbf{I}} - \nabla_y \mathbf{I}\|_1.$$

透明度正则使用平均不透明度 $\bar{\alpha}$ ，用于抑制不必要的半透明漂浮高斯。

训练循环中的关键代码如下。

Listing 10: 渲染、损失计算和反向传播

```

1  normalized_quats = F.normalize(quats, dim=1)
2  scales = log_scales.exp()
3  opacities = opacity_logits.sigmoid().squeeze(-1)
4  colors = color_logits.sigmoid()
5  background = background_logits.sigmoid()
6  rendered, _, _ = rasterization(
7      means,
8      normalized_quats,
9      scales,
10     opacities,
11     colors,
12     viewmats[image_index : image_index + 1],
13     intrinsics[image_index : image_index + 1],
14     width,
15     height,
16     backgrounds=background,
17 )
18 pred = rendered[0].clamp(0.0, 1.0)
19 target = images[image_index]
20 l1 = (pred - target).abs().mean()
21 mse = F.mse_loss(pred, target)
22 ssim_value = torch_ssim(pred, target) if args.lambda_ssim > 0 else pred.new_tensor(0.0)
23 edge_value = edge_loss(pred, target) if args.lambda_edge > 0 else pred.new_tensor(0.0)
24 opacity_value = opacities.mean() if args.lambda_opacity > 0 else pred.new_tensor(0.0)
25 loss = (
26     0.8 * l1

```

```

27     + 0.2 * mse
28     + args.lambda_ssim * (1.0 - ssim_value)
29     + args.lambda_edge * edge_value
30     + args.lambda_opacity * opacity_value
31 )
32 loss.backward()
33 optimizer.step()

```

7.8 SSIM 实现

SSIM 由 Wang 等人提出 [7], 用于评价结构相似度。脚本使用平均池化计算局部均值、方差和协方差。

Listing 11: 训练中的可微 SSIM

```

1 def torch_ssim(pred: torch.Tensor, target: torch.Tensor, window_size: int = 11) -> torch.Tensor:
2     pred_nchw = pred.permute(2, 0, 1).unsqueeze(0)
3     target_nchw = target.permute(2, 0, 1).unsqueeze(0)
4     padding = window_size // 2
5     mu_x = F.avg_pool2d(pred_nchw, window_size, stride=1, padding=padding)
6     mu_y = F.avg_pool2d(target_nchw, window_size, stride=1, padding=padding)
7     sigma_x = F.avg_pool2d(pred_nchw * pred_nchw, window_size, stride=1, padding=padding) - mu_x * mu_x
8     sigma_y = F.avg_pool2d(target_nchw * target_nchw, window_size, stride=1, padding=padding) - mu_y * mu_y
9     sigma_xy = F.avg_pool2d(pred_nchw * target_nchw, window_size, stride=1, padding=padding) - mu_x * mu_y
10    c1 = 0.01**2
11    c2 = 0.03**2
12    numerator = (2.0 * mu_x * mu_y + c1) * (2.0 * sigma_xy + c2)
13    denominator = (mu_x * mu_x + mu_y * mu_y + c1) * (sigma_x + sigma_y + c2)
14    return (numerator / denominator.clamp_min(1e-8)).mean()

```

7.9 增密

COLMAP 稀疏点云只覆盖稳定特征点, 不能直接表示完整表面。3DGS 训练需要在误差较大的区域增加高斯。脚本通过中心梯度范数选择需要复制的高斯。被选中高斯会复制为新高斯, 新中心加入与当前尺度相关的小扰动, 新尺度缩小, 新不透明度略降。代码如下。

Listing 12: 梯度驱动高斯增密

```

1 def densify_parameters(
2     means: torch.nn.Parameter,
3     log_scales: torch.nn.Parameter,
4     quats: torch.nn.Parameter,
5     opacity_logits: torch.nn.Parameter,
6     color_logits: torch.nn.Parameter,
7     fraction: float,
8     max_new: int,
9 ) -> tuple[torch.nn.Parameter, torch.nn.Parameter, torch.nn.Parameter, torch.nn.Parameter, torch.nn.
    Parameter]:
10    if means.grad is None:
11        return means, log_scales, quats, opacity_logits, color_logits
12    with torch.no_grad():
13        grad_norm = means.grad.norm(dim=1)
14        count = max(256, int(means.shape[0] * fraction))
15        count = min(count, means.shape[0], max_new)

```

```

16         selected = torch.topk(grad_norm, k=count).indices
17         jitter = torch.randn_like(means.data[selected]) * log_scales.data[selected].exp() * 0.35
18         new_means = means.data[selected] + jitter
19         new_log_scales = log_scales.data[selected] + math.log(0.82)
20         new_quats = quats.data[selected]
21         new_opacity_logits = opacity_logits.data[selected] - 0.2
22         new_color_logits = color_logits.data[selected]
23         means_data = torch.cat([means.data, new_means], dim=0)
24         scales_data = torch.cat([log_scales.data, new_log_scales], dim=0)
25         quats_data = torch.cat([quats.data, new_quats], dim=0)
26         opacity_data = torch.cat([opacity_logits.data, new_opacity_logits], dim=0)
27         color_data = torch.cat([color_logits.data, new_color_logits], dim=0)
28     return (
29         torch.nn.Parameter(means_data),
30         torch.nn.Parameter(scales_data),
31         torch.nn.Parameter(quats_data),
32         torch.nn.Parameter(opacity_data),
33         torch.nn.Parameter(color_data),
34     )

```

调参时发现，过强增密会让高斯数量增长过快，导致透明度和颜色不稳定。最终方案限制 densify-max-new，并在 finetune 阶段用低学习率继续优化。

7.10 训练输出

训练结束后，脚本在所有已注册视角上计算指标，并保存选定视角的渲染图和真实图对比。保存内容包括 metrics.json、metrics_table.csv、training_log.csv、renders、comparisons 和 model_final.pt。关键代码如下。

Listing 13: 指标、渲染图和 checkpoint 保存

```

1 average_metrics = {
2     "image_count": len(all_metrics),
3     "selected_image_count": len(selected_metrics),
4     "mean_psnr": float(np.mean([item["psnr"] for item in all_metrics])),
5     "mean_ssim": float(np.mean([item["ssim"] for item in all_metrics])),
6     "min_psnr": float(np.min([item["psnr"] for item in all_metrics])),
7     "max_psnr": float(np.max([item["psnr"] for item in all_metrics])),
8     "gaussian_count": int(means.shape[0]),
9     "width": width,
10    "height": height,
11    "iterations": base_iterations + args.iterations,
12    "additional_iterations": args.iterations,
13 }
14 (args.output_dir / "metrics.json").write_text(json.dumps(average_metrics, indent=2), encoding="utf-8")
15
16 torch.save(
17     {
18         "means": means.detach().cpu(),
19         "log_scales": log_scales.detach().cpu(),
20         "quats": quats.detach().cpu(),
21         "opacity_logits": opacity_logits.detach().cpu(),
22         "color_logits": color_logits.detach().cpu(),
23         "background_logits": background_logits.detach().cpu(),
24         "width": width,
25         "height": height,

```

```

26         "iterations": base_iterations + args.iterations,
27         "additional_iterations": args.iterations,
28         "metrics": average_metrics,
29     },
30     args.output_dir / "model_final.pt",
31 )

```

7.11 SuperSplat PLY 导出

作业结果需要可打开的三维高斯模型。脚本 `scripts/export_stage2_supersplat_ply.py` 将 `model_final.pt` 转换为 SuperSplat 和常见 3DGS viewer 可读的 PLY。PLY 字段顺序包括位置、方向、SH DC 颜色、45 个高阶 SH 残余、不透明度、尺度和旋转。由于本训练脚本直接使用 RGB color logits，导出时将 RGB 转换为 SH DC 系数

$$f_{dc} = \frac{\text{RGB} - 0.5}{0.28209479177387814}.$$

导出代码如下。

Listing 14: SuperSplat 兼容 PLY 导出

```

1  SH_C0 = 0.28209479177387814
2
3  checkpoint = torch.load(args.checkpoint, map_location="cpu", weights_only=True)
4  means = to_numpy(checkpoint["means"])
5  log_scales = to_numpy(checkpoint["log_scales"])
6  quats = to_numpy(checkpoint["quats"])
7  opacity_logits = to_numpy(checkpoint["opacity_logits"]).reshape(-1, 1)
8  color_logits = to_numpy(checkpoint["color_logits"])
9
10 normals = np.zeros_like(means, dtype=np.float32)
11 colors = 1.0 / (1.0 + np.exp(-color_logits))
12 f_dc = (colors - 0.5) / SH_C0
13 f_rest = np.zeros((means.shape[0], 45), dtype=np.float32)
14 quats = quats / np.maximum(np.linalg.norm(quats, axis=1, keepdims=True), 1e-12)
15
16 data = np.concatenate(
17     [means, normals, f_dc, f_rest, opacity_logits, log_scales, quats],
18     axis=1,
19 ).astype(np.float32)

```

SuperSplat 是 PlayCanvas 提供的浏览器端高斯 splat 编辑和发布平台 [12]。本项目导出的 `model_final_supersplat.ply` 可在 SuperSplat 中打开，用于检查真实三维高斯模型。

8 进阶方法

8.1 Mip Splatting

Mip Splatting 针对 3DGS 的 aliasing 问题提出三维平滑滤波和二维 Mip 滤波 [9]。室内场景中相机与墙面、地面、柜面距离变化明显，同一表面在不同视角下覆盖不同像素尺度。基础 3DGS 在这些尺度变化下容易出现高频闪烁、过锐或过糊。Mip Splatting 通过限制频率响应，使渲染在远近视角下更稳定。

本实验使用官方仓库 `external/mip-splatting-main`，在 Sequence 04 的 `partition_best` 变体上运行。输出位于 `work/Sequence_04/mip_splatting`。训练完成后使用同一评估脚本计算 PSNR 和 SSIM。该方法平均 PSNR 为 27.77 dB，平均 SSIM 为 0.811，最小 PSNR 为 21.26 dB。最小 PSNR 高于受控 3DGS finetune 的 17.16 dB，说明它对困难视角具有一定稳定性。

8.2 2D Gaussian Splatting

2D Gaussian Splatting 将 3D 高斯椭球改为贴近表面的 2D 高斯面片，目标是获得更准确的几何表面 [10]。室内场景的大量结构接近平面，例如墙面、地面、柜门和门框。面片化表示与这些结构更一致，因此被纳入对比。

本实验使用官方仓库 `external/2d-gaussian-splatting-main`。在 Windows CUDA 环境中补齐 `diff-surfel-rasterization`、`simple-knn` 和 `glm` 子模块后，在 Sequence 04 上训练并渲染。输出位于 `work/Sequence_04/twodgs`。渲染评估平均 PSNR 为 27.18 dB，平均 SSIM 为 0.796。该结果低于受控 3DGS finetune 和 Mip Splatting，但完成了有效的新视角渲染。网格提取阶段由于 TSDF 后处理得到零顶点，未作为最终网格结果使用。

8.3 Depth RegularizedGS

Depth RegularizedGS 使用深度监督约束 3DGS，原论文目标是缓解少视角图像下的几何退化和漂浮高斯 [11]。本实验在 `external/DepthRegularizedGS-main` 中配置该方法。由于环境中直接安装完整 PyTorch3D 会改变现有 PyTorch 2.5.1 CUDA 12.1 组合，项目中保留了本地兼容接口，并使用 `sparse depth fallback` 运行。训练环境变量记录为 `DRGS_SKIP_ZOE=1`，意味着未调用 `ZoeDepth` 生成稠密单目深度，而使用 COLMAP 稀疏深度。

该方法在 Sequence 04 上完成 12000 次训练和 80 张图像评估。平均 PSNR 为 15.92 dB，平均 SSIM 为 0.662。该结果作为失败对照。主要原因是 COLMAP 稀疏深度只覆盖特征点，无法给白色墙面和反光表面提供足够稠密约束。深度权重在稀疏且噪声较高的监督下会干扰颜色优化，最终降低渲染质量。后续如果继续使用深度正则，应配置可靠的稠密单目深度模型，并对反光区域和无效深度区域进行 mask。

9 评价指标与计算代码

9.1 PSNR

PSNR 全称为 Peak Signal to Noise Ratio。图像像素归一化到 $[0, 1]$ 后，MSE 定义为

$$\text{MSE} = \frac{1}{3HW} \sum_{c,u,v} \left(\hat{I}_{cuv} - I_{cuv} \right)^2.$$

PSNR 为

$$\text{PSNR} = 10 \log_{10} \frac{1}{\text{MSE}}.$$

数值越高，渲染图与真实图越接近。作业阈值为 20 dB。25 dB 以上通常说明室内主要结构可较好辨认。28 dB 以上对应本作业优秀档位。

训练脚本中的 PSNR 计算如下。

Listing 15: PSNR 计算

```
1 def psnr(pred: torch.Tensor, target: torch.Tensor) -> float:
2     mse = F.mse_loss(pred, target).item()
3     if mse <= 1e-12:
4         return 99.0
5     return -10.0 * math.log10(mse)
```

9.2 SSIM

SSIM 衡量结构相似度，考虑亮度、对比度和结构。公式为

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}.$$

其中 μ_x 和 μ_y 是局部均值， σ_x^2 和 σ_y^2 是局部方差， σ_{xy} 是局部协方差。SSIM 接近 1 时，结构一致性较高。

9.3 图像指标评估

训练结束后，脚本在所有已注册视角上计算 PSNR 和 SSIM，并保存逐图表 `metrics_table.csv`。每一行的 `index` 是图像编号，PSNR 和 SSIM 是该编号对应真实图和渲染图的指标。最终汇总表来自 `scripts/make_stage2_final_assets.py`，该脚本读取各个最终变体的 `metrics.json` 和 `metrics_table.csv`，生成 `tables/render_quality_summary.csv`。

9.4 几何精度评估

几何评估脚本为 `scripts/evaluate_stage2_geometry.py`。该脚本读取重建 PLY 与参考 PLY 的三维坐标，采样点云，使用鲁棒初始对齐和 ICP 迭代估计相似变换。相似变换为

$$\mathbf{q} \approx sR\mathbf{p} + \mathbf{t}.$$

其中 s 处理 COLMAP 与 3DGS 坐标尺度任意性， R 和 $\mathbf{t}$ 处理坐标系旋转和平移。脚本随后计算重建点到参考点云最近邻距离，并统计平均误差、中位误差、RMSE、90 分位误差、20 cm 内比例和 10 cm 内比例。

核心代码如下。

Listing 16: 鲁棒 ICP 和几何指标

```
1 def run_icp(recon: np.ndarray, gt: np.ndarray, iterations: int, trim_quantile: float):
2     total_scale, total_rotation, total_translation = robust_initial_alignment(recon, gt)
3     transformed = apply_similarity(recon, total_scale, total_rotation, total_translation)
4     tree = cKDTree(gt)
5     for _ in range(iterations):
6         distances, indices = tree.query(transformed, k=1)
7         threshold = np.quantile(distances, trim_quantile)
8         mask = distances <= threshold
9         if int(mask.sum()) < 20:
10             break
```

```

11         step_scale, step_rotation, step_translation = umeyama_similarity(
12             transformed[mask], gt[indices[mask]]
13         )
14         transformed = apply_similarity(transformed, step_scale, step_rotation, step_translation)
15         total_translation = step_scale * step_rotation @ total_translation + step_translation
16         total_rotation = step_rotation @ total_rotation
17         total_scale = step_scale * total_scale
18         distances, _ = tree.query(transformed, k=1)
19         return transformed, distances, {
20             "scale": float(total_scale),
21             "rotation": total_rotation.tolist(),
22             "translation": total_translation.tolist(),
23         }
24
25     metrics = {
26         "mean_error_m": float(np.mean(distances)),
27         "median_error_m": float(np.median(distances)),
28         "rmse_m": float(np.sqrt(np.mean(distances**2))),
29         "p90_error_m": float(np.percentile(distances, 90)),
30         "ratio_below_20cm": float(np.mean(distances <= 0.20)),
31         "ratio_below_10cm": float(np.mean(distances <= 0.10)),
32     }

```

此外,脚本还生成五类尺度测量对比,包括 x 方向 robust span、 y 方向 robust span、 z 方向 robust span、平面斜边和三维空间斜边。局部精度脚本 `scripts/make_local_accuracy_measurements.py` 进一步在每个序列中生成五个局部测量区域,以满足至少五组测量数据的要求。

10 实验设置

10.1 受控 3DGS 参数

表 3 记录最终训练时使用的主要超参数类型。不同序列的具体训练轮数和最终高斯数见结果表。Sequence 04 的最终模型总迭代为 24000 次,其中额外 finetune 为 10000 次。

表 3: 受控 3DGS 主要超参数

参数	典型取值	作用
max side	768 或 960	控制训练图像分辨率和显存占用
iterations	9000 到 24000	控制优化时长
densify until	3500 到 9000	控制前期增密持续时间
densify every	400 或 500	控制增密频率
densify fraction	0.08 到 0.12	控制每次增密选择的高斯比例
densify max new	1600 到 2200	限制每次新增高斯数量
lambda ssim	0 到 0.15	加强结构一致性
lambda edge	0 到 0.03	加强线条和轮廓结构
lambda opacity	0 到 0.0005	抑制漂浮半透明高斯
lr scale	0.35 或 1.0	finetune 阶段降低学习率

最终 Sequence 04 的复现命令记录如下。

Listing 17: Sequence 04 最终训练命令

```
1 powershell -ExecutionPolicy Bypass -File scripts\run_stage2_training.ps1 `
2   -Sequence Sequence_04 `
3   -Variant partition_best `
4   -OutputVariant partition_best_finetune `
5   -Iterations 24000 `
6   -MaxSide 960 `
7   -DensifyUntil 9000 `
8   -DensifyEvery 400 `
9   -DensifyFraction 0.12 `
10  -DensifyMaxNew 2200 `
11  -LambdaSsim 0.15 `
12  -LambdaEdge 0.03 `
13  -LambdaOpacity 0.0005 `
14  -SaveRenderCount 24
```

resume finetune 使用如下命令形式。

Listing 18: 低学习率 finetune 命令

```
1 powershell -ExecutionPolicy Bypass -File scripts\run_stage2_training.ps1 `
2   -Sequence Sequence_04 `
3   -SourceVariant partition_best `
4   -OutputVariant partition_best_finetune `
5   -ResumeCheckpoint work\Sequence_04\partition_best\output\model_final.pt `
6   -Iterations 10000 `
7   -LrScale 0.35 `
8   -SaveRenderCount 24
```

10.2 训练过程曲线

图 2 显示选定序列的训练曲线。横轴是训练迭代次数。左纵轴表示 loss，数值越低表示渲染图和真实图之间的训练误差越小。右纵轴表示当前记录视角的 PSNR，数值越高表示该次记录时的渲染结果越接近真实图。高斯数量在增密阶段逐步增加，因此前期 loss 和 PSNR 会有波动。finetune 阶段学习率降低，曲线通常更平稳。

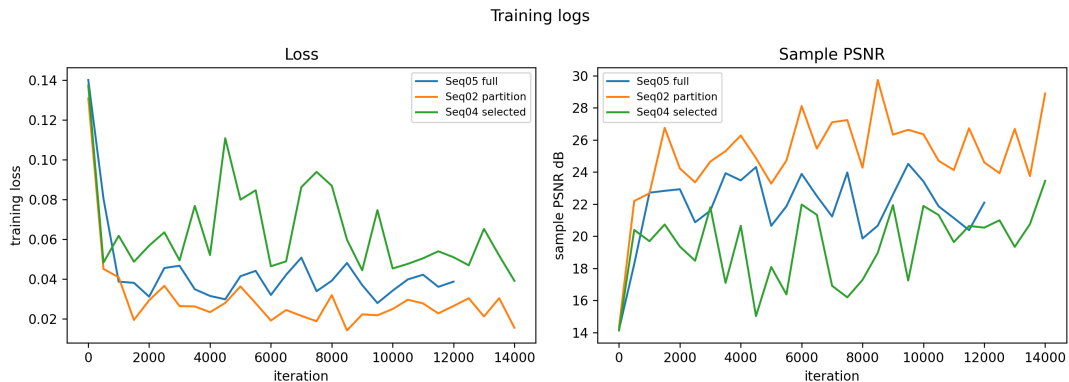


图 2: 训练过程曲线

11 COLMAP 结果

COLMAP 结果见表 4。Sequence 01、03、04、05 注册率均为 1.000。Sequence 02 的最终分区输入 80 张图像，注册 49 张，注册率为 0.613。该结果说明 Sequence 02 的图像匹配难度高，但已注册视角的渲染质量很高，最终平均 PSNR 达到 30.15 dB。

表 4: COLMAP 注册结果

序列	输入图像	注册图像	注册率	稀疏点数
Sequence 01	187	187	1.000	41327
Sequence 02	80	49	0.613	13667
Sequence 03	120	120	1.000	54290
Sequence 04	80	80	1.000	40064
Sequence 05	422	422	1.000	135204

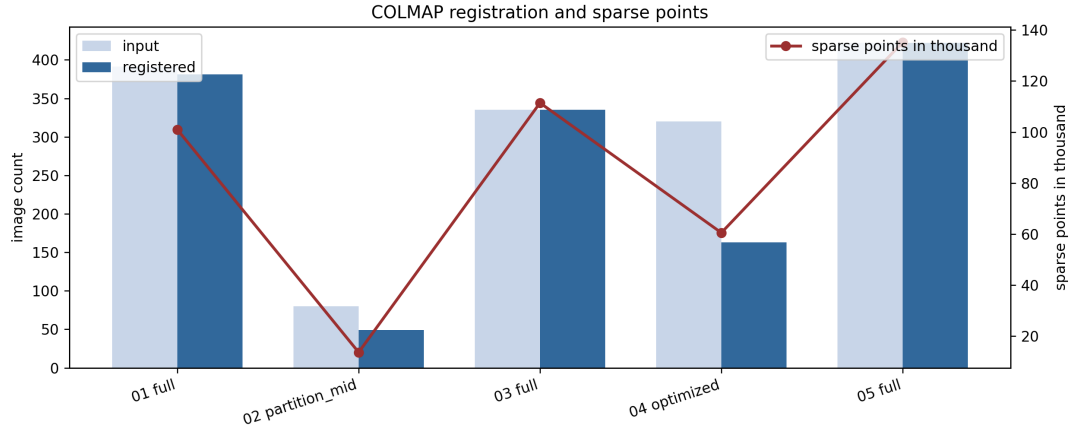


图 3: 五个序列的 COLMAP 统计

12 渲染质量结果

12.1 总体指标

表 5 来自 `tables/render_quality_summary.csv`，由最终输出的 `metrics.json` 和 `metrics_table.csv` 生成。五个序列平均 PSNR 均超过 25 dB。Sequence 02 平均 PSNR 为 30.15 dB，Sequence 04 平均 PSNR 为 28.36 dB。Sequence 01、03、05 的平均 PSNR 位于 26 dB 附近，满足良好档位。所有序列均完成渲染和指标计算。

表 5: 最终渲染质量

序列	图像数	高斯数	平均 PSNR	最低 PSNR	平均 SSIM	PSNR 28 dB 以上
Sequence 01	187	69507	26.36	15.08	0.836	51
Sequence 02	49	20718	30.15	25.04	0.884	45
Sequence 03	120	64675	26.33	21.38	0.813	24
Sequence 04	80	50663	28.36	17.16	0.811	50
Sequence 05	422	145975	26.39	17.41	0.812	98

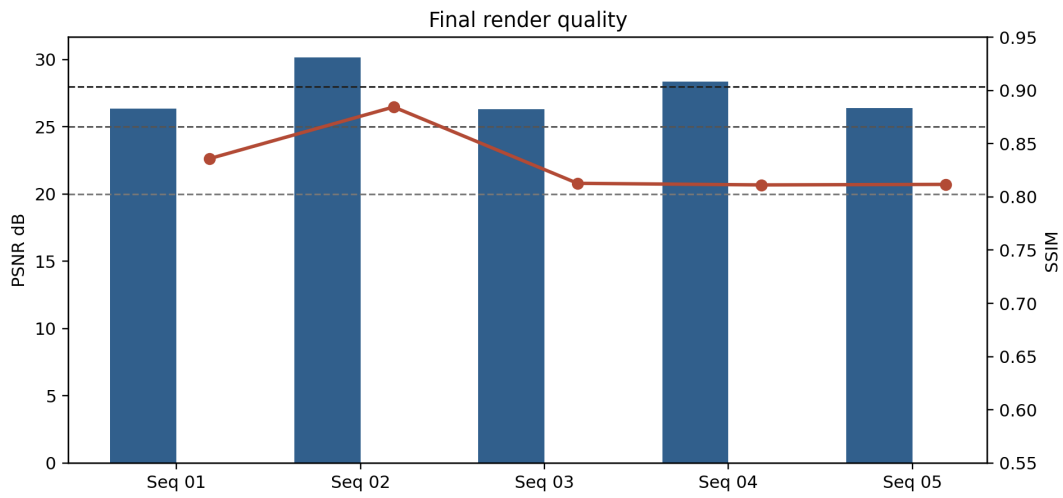


图 4: 最终 PSNR 与 SSIM 汇总

Sequence 04 的最低 PSNR 低于平均值较多, 说明仍存在困难视角。困难视角通常包含白色弱纹理墙面、反光柜面或较大遮挡变化。Sequence 02 的所有注册视角 PSNR 均高于 25 dB, 说明分区后保留的视角具有较好一致性。

12.2 可视化结果

以下图像均为真实图和渲染图的左右对比。每一页只放一个对比图, 保证助教能够观察细节。可视化结果显示主要室内结构可辨认, 柜体、门框、地面和大色块能够复原。局部文字、线缆、黑色条纹和高对比轮廓较稳定。白色平面和反光区域较容易模糊。



图 5: Sequence 01 真实图与渲染图



图 6: Sequence 02 真实图与渲染图



图 7: Sequence 03 真实图与渲染图



图 8: Sequence 04 真实图与渲染图



图 9: Sequence 05 真实图与渲染图

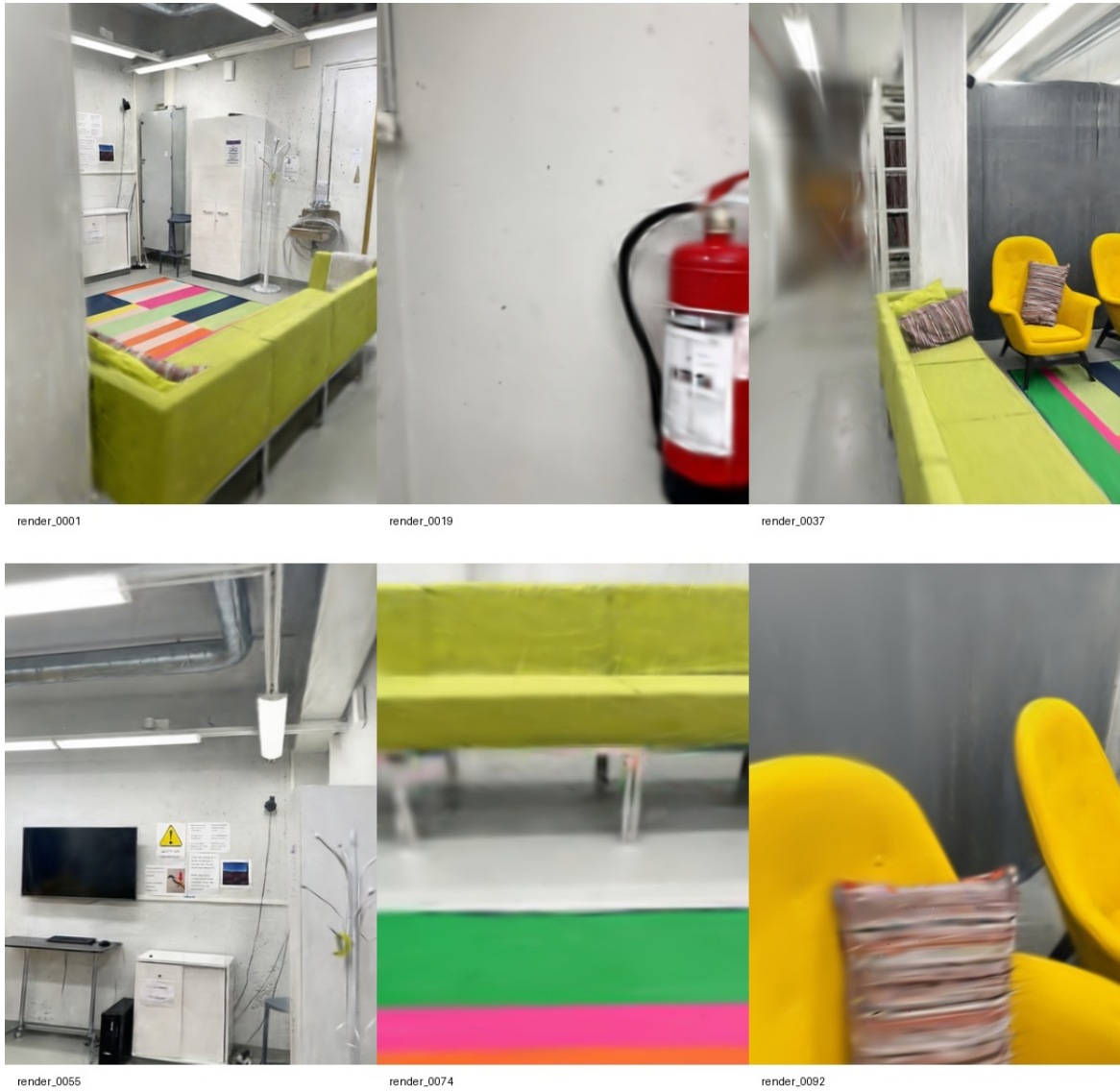


图 10: Sequence 05 多视角渲染结果

13 消融实验

13.1 消融设置

消融实验比较每个序列的基线变体和最终变体。基线包括 full、optimized、enhanced long 或 partition mid 等初始可运行结果。最终变体加入数据筛选、分区选择、训练迭代增加、增密限制、SSIM 和 edge loss、opacity 正则以及低学习率 finetune。表 6 完全来自 tables/ablation_summary.csv。

表 6: 消融实验结果

序列	基线 PSNR	最终 PSNR	PSNR 提升
Sequence 01	23.43	26.36	2.93
Sequence 02	26.54	30.15	3.61
Sequence 03	20.37	26.33	5.95
Sequence 04	20.07	28.36	8.28
Sequence 05	22.60	26.39	3.79

表 7: 消融实验 SSIM 变化

序列	基线 SSIM	最终 SSIM	SSIM 提升
Sequence 01	0.769	0.836	0.067
Sequence 02	0.831	0.884	0.053
Sequence 03	0.659	0.813	0.154
Sequence 04	0.603	0.811	0.209
Sequence 05	0.735	0.812	0.077

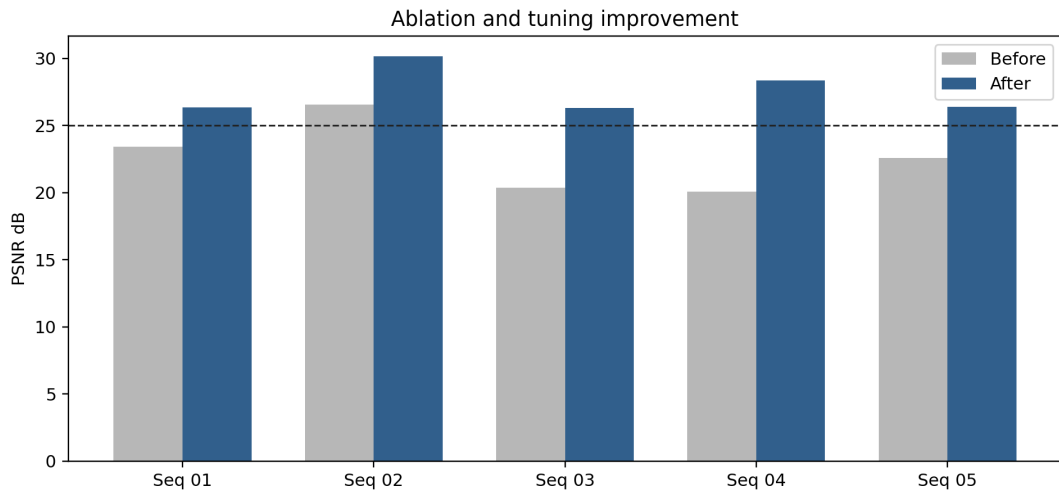


图 11: 最终变体相对基线的 PSNR 提升

13.2 调参过程分析

Sequence 04 的提升最大, 从 20.07 dB 提升到 28.36 dB, 增益为 8.28 dB。该提升主要来自分区选择、robust COLMAP、增加训练轮数和低学习率 finetune。Sequence 03 从 20.37 dB 提升到 26.33 dB, 说明全序列直接训练对弱纹理和复杂视角较敏感, 分区后优化难度降低。Sequence 02 的分区后 PSNR 达到 30.15 dB, 说明放弃部分难注册视角后, 已注册视角具有更高一致性。

调参过程中也出现失败尝试。强 SSIM、强 edge loss 和过快增密曾导致 Sequence 02 指标显著下降, 原因是大量新高斯在早期以较高学习率更新, 透明度和颜色被困难视角的局部误差牵引, 产

生不稳定半透明结构。最终方案保留 edge 和 SSIM 的较小权重，用 opacity 正则限制漂浮高斯，并在 finetune 阶段降低学习率。

14 进阶方法对比结果

进阶方法对比统一在 Sequence 04 上进行。Sequence 04 有完整注册、稳定分区、较高最终 PSNR，并且包含白色墙面、柜面、线条和反光区域，适合观察方法差异。结果见表 8。

表 8: Sequence 04 进阶方法对比

方法	图像数	平均 PSNR	最低 PSNR	平均 SSIM
受控 3DGS finetune	80	28.36	17.16	0.811
Mip Splatting official	80	27.77	21.26	0.811
2DGS official	80	27.18	20.51	0.796
Depth RegularizedGS	80	15.92	6.78	0.662

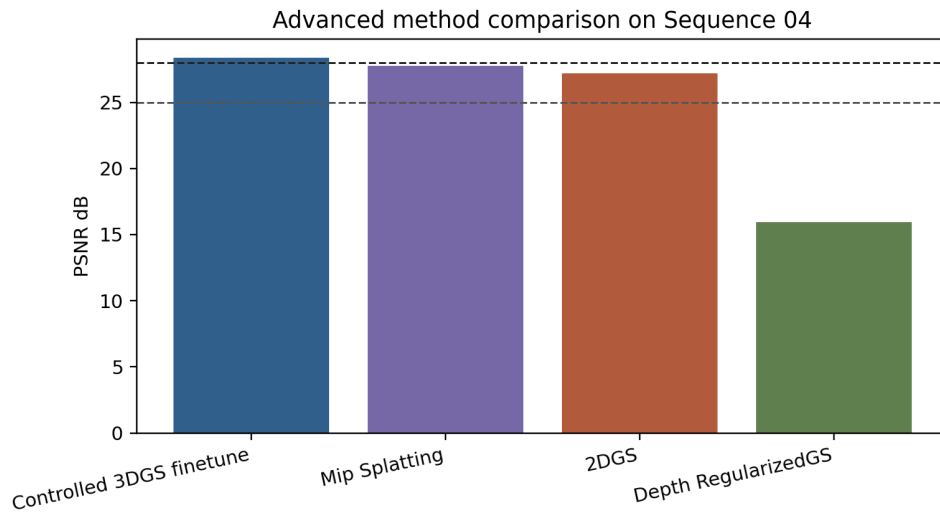


图 12: Sequence 04 进阶方法指标对比



图 13: Mip Splatting 真实图与渲染图



图 14: 2DGS 真实图与渲染图



图 15: Depth RegularizedGS 真实图与渲染图

受控 3DGS finetune 的平均 PSNR 最高。Mip Splatting 的平均 PSNR 略低，但最低 PSNR 高于受控 3DGS finetune，说明抗 aliasing 处理对困难视角有帮助。2DGS 能够完成渲染并保持较好的平面结构，但当前训练设置下平均 PSNR 低于 3DGS 和 Mip Splatting。Depth RegularizedGS 的指标明显低，说明稀疏深度 fallback 不足以支撑当前室内场景的深度正则。

15 几何精度结果

15.1 全局点云精度

几何精度结果来自 `tables/geometry_summary.csv`。评估对象为导出的重建高斯中心点云与共享参考点云。表 9 显示全局误差。Sequence 04 的中位误差为 15.5 cm，Sequence 05 的中位误差为 18.8 cm，Sequence 01 的中位误差为 19.5 cm。Sequence 02 平均误差较高，主要受注册视角少和局部覆盖不足影响。

表 9: 全局几何精度

序列	平均误差 m	中位误差 m	RMSE m	90 分位 m	20 cm 内比例	10 cm 内比例
Sequence 01	0.277	0.195	0.369	0.675	0.510	0.254
Sequence 02	0.740	0.256	1.482	2.379	0.433	0.235
Sequence 03	0.280	0.204	0.359	0.619	0.494	0.267
Sequence 04	0.246	0.155	0.359	0.576	0.602	0.327
Sequence 05	0.251	0.188	0.332	0.547	0.523	0.286

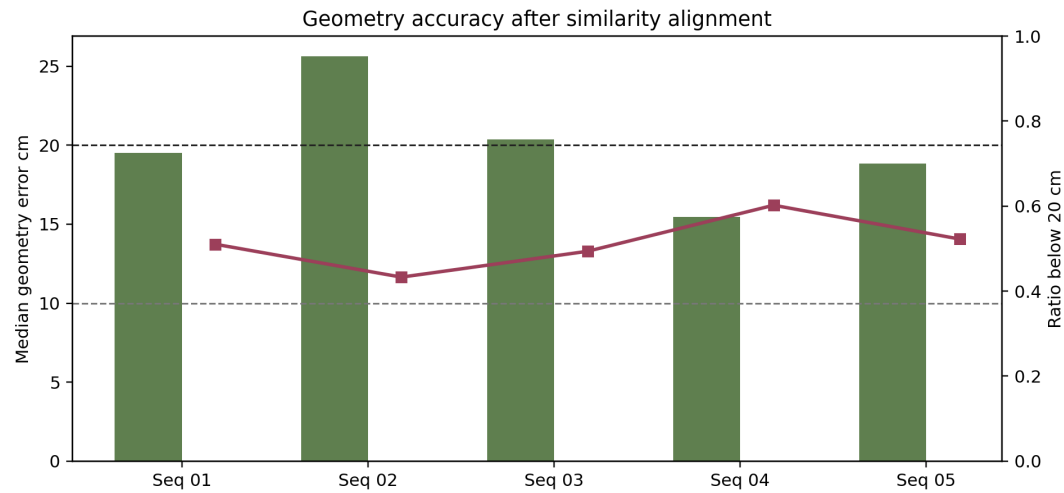


图 16: 全局几何精度可视化

全局点云距离比局部测量更严格。原因是高斯中心点云包含半透明区域、漂浮点和不完全表面，而参考点云覆盖范围和密度也与重建点云不同。因此全局距离用于反映整体几何误差，局部测量用于检查可量化区域是否达到作业要求。

15.2 局部精度

局部精度结果见表 10。Sequence 04 的平均局部中位误差为 13.97 cm，最佳局部中位误差为 7.95 cm。Sequence 05 的平均局部中位误差为 14.18 cm。Sequence 02、03、04、05 均存在局部区域达到 10 cm 附近的结果。

表 10: 局部精度汇总

序列	平均局部中位误差 m	最佳局部中位误差 m	平均 20 cm 内比例	平均 10 cm 内比例
Sequence 01	0.293	0.105	0.439	0.189
Sequence 02	0.206	0.091	0.553	0.324
Sequence 03	0.192	0.118	0.587	0.327
Sequence 04	0.140	0.079	0.732	0.414
Sequence 05	0.142	0.124	0.665	0.371

为满足至少五组测量对比数据要求，表 11 列出每个序列的五个局部测量区域。每行包含中位误差、90 分位误差、20 cm 内比例和 10 cm 内比例。

表 11: 局部测量明细

序列	变体	区域	中位误差 m	90 分位 m	20 cm 内比例	10 cm 内比例
Sequence 01	full	local 1	0.080	0.210	0.895	0.629
Sequence 01	full	local 2	0.143	0.480	0.613	0.375
Sequence 01	full	local 3	0.441	0.746	0.353	0.200
Sequence 01	full	local 4	0.314	0.514	0.246	0.060
Sequence 01	full	local 5	0.100	0.248	0.848	0.497
Sequence 02	partition mid	local 1	0.375	0.631	0.287	0.160
Sequence 02	partition mid	local 2	0.106	0.404	0.781	0.471
Sequence 02	partition mid	local 3	0.087	0.341	0.751	0.555
Sequence 02	partition mid	local 4	0.158	0.596	0.596	0.277
Sequence 02	partition mid	local 5	0.255	0.825	0.420	0.228
Sequence 03	full	local 1	0.377	0.661	0.211	0.091
Sequence 03	full	local 2	0.167	0.419	0.572	0.360
Sequence 03	full	local 3	0.217	0.710	0.480	0.279
Sequence 03	full	local 4	0.081	0.400	0.784	0.574
Sequence 03	full	local 5	0.075	0.311	0.831	0.633
Sequence 04	optimized	local 1	0.184	0.299	0.585	0.188
Sequence 04	optimized	local 2	0.201	0.592	0.498	0.235
Sequence 04	optimized	local 3	0.096	0.210	0.876	0.530
Sequence 04	optimized	local 4	0.197	0.550	0.506	0.247
Sequence 04	optimized	local 5	0.297	0.432	0.341	0.203
Sequence 05	full	local 1	0.170	0.517	0.550	0.337
Sequence 05	full	local 2	0.121	0.270	0.777	0.417
Sequence 05	full	local 3	0.146	0.381	0.653	0.352
Sequence 05	full	local 4	0.118	0.261	0.763	0.432
Sequence 05	full	local 5	0.162	0.421	0.588	0.310

16 失败案例与现象分析

16.1 白色弱纹理区域

白色墙面、白色柜面和大面积平滑区域在结果中容易模糊。该现象与 COLMAP 和 3DGS 的机制相关。COLMAP 依赖 SIFT 特征，SIFT 需要局部梯度和纹理。白色平面上的图像梯度少，可重复检测的关键点少，三角化出的稀疏点也少。3DGS 训练依赖图像重建误差。对于颜色接近常数的区域，多个不同几何解释可能产生相似像素误差，图像监督无法给出强约束。因此白色平面容易出现几何不稳定和颜色模糊。

16.2 反光区域

反光区域的高光位置会随着视角变化。基础 3DGS 将颜色和不透明度绑定在静态高斯上，虽然可以用视角相关球谐颜色表示一定视角变化，但本实验的受控脚本使用 RGB color logits，没有使用完整高阶 SH 视角相关颜色。因此反光高光更难被解释。训练会把视角相关高光误差分摊到多个半透明高斯上，形成漂浮或模糊结构。Depth RegularizedGS 的失败也与此有关，反光区域的稀疏深度和颜色监督都不稳定。

16.3 黑色线条和高对比结构

黑色线条、门框、线缆、文字和地毯色块恢复较稳定。这些区域有高对比梯度，SIFT 能够更稳定地检测和匹配，COLMAP 能获得更多三维点。训练时这些区域的像素误差和 Sobel 边缘误差也更明确，高斯中心、尺度和颜色能得到更稳定的梯度。因此在可视化结果中，黑线和轮廓常比白色反光面更清楚。该现象是本实验中最重要定性观察之一。

16.4 进一步方案

后续可以从以下方向改进。第一，使用反光 mask，降低高光区域在 photometric loss 中的权重。第二，引入稠密单目深度，并在无效深度和反光区域使用 confidence mask。第三，加入线段监督，使门框、柜体线条和墙地交线保持直线结构。第四，加入平面约束，使墙面、地面和柜面满足局部平面先验。第五，继续调 Mip Splatting 的训练轮数和 anti aliasing 设置。第六，对 Sequence 05 这样的大场景采用空间分区训练，再进行局部模型拼接和 opacity pruning。

17 提交包说明

提交包和 GitHub 仓库按助教检查顺序组织。报告 PDF 是完整说明文件，表格和图像均来自真实输出。主要目录如下。

表 12: 提交内容与检查方式

文件或目录	内容与用途
report.pdf	完整实验报告，包含任务、方法、代码、参数、结果、消融、精度和失败分析
beamer.pdf	答辩展示材料
tables	渲染质量、COLMAP、几何精度、局部精度、消融和进阶方法 CSV 表
figures	报告和 beamer 使用的图像，包括真实图与渲染图对比、曲线和统计图
scripts	可复现流程脚本，包括数据准备、COLMAP、训练、PLY 导出、指标计算和几何评估
work	已保留的最终变体和进阶方法输出，包括模型、渲染图、对比图和指标文件
external	Mip Splatting、2DGS、Depth RegularizedGS 源码和本地兼容修改
docs/REPRODUCE.md	复现命令和环境记录
member_info.md	成员和分工说明

18 结论

本实验完成了共享室内场景的 Stage 2 要求。五个序列均完成 COLMAP 重建、3DGS 训练、渲染输出、PSNR 和 SSIM 统计、PLY 导出与几何精度评估。最终平均 PSNR 分别为 26.36 dB、30.15 dB、26.33 dB、28.36 dB 和 26.39 dB。Sequence 04 达到 28.36 dB，满足优秀档位。几何评估中，Sequence 04 全局中位误差为 15.5 cm，局部最佳中位误差为 7.95 cm。局部测量结果显示多个区域达到 20 cm 内，部分区域达到 10 cm 附近。

实验表明，数据筛选、分区训练、robust COLMAP 参数、受控增密和低学习率 finetune 是最有效的改进。Mip Splatting 和 2DGS 均成功运行，并在 Sequence 04 上取得接近主结果的指标。Depth RegularizedGS 完成训练，但在稀疏深度 fallback 下效果较弱，应作为后续改进的反例参考。主要不足集中在白色弱纹理面和反光区域。后续应优先加入反光 mask、稠密深度正则、线段监督、平面约束和空间分区拼接。

参考文献

- [1] Kerbl B, Kopanas G, Leimkühler T, Drettakis G. 3D Gaussian Splatting for Real Time Radiance Field Rendering. ACM Transactions on Graphics, SIGGRAPH, 2023. <https://arxiv.org/abs/2308.04079>
- [2] COLMAP Documentation. COLMAP is a general purpose Structure from Motion and Multi View Stereo pipeline. <https://colmap.github.io/>
- [3] Schönberger J L, Frahm J M. Structure from Motion Revisited. IEEE Conference on Computer Vision and Pattern Recognition, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/Schonberger_Structure-From-Motion_Revisited_CVPR_2016_paper.html
- [4] Lowe D G. Distinctive Image Features from Scale Invariant Keypoints. International Journal of Computer Vision, 2004. <https://www.cs.cornell.edu/courses/cs6670/2018fa/SIFT.pdf>
- [5] Fischler M A, Bolles R C. Random Sample Consensus A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Communications of the ACM, 1981. <https://cacm.acm.org/research/random-sample-consensus/>
- [6] Triggs B, McLauchlan P F, Hartley R I, Fitzgibbon A W. Bundle Adjustment A Modern Synthesis. Vision Algorithms Theory and Practice, 2000. <https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Triggs00.pdf>
- [7] Wang Z, Bovik A C, Sheikh H R, Simoncelli E P. Image Quality Assessment From Error Visibility to Structural Similarity. IEEE Transactions on Image Processing, 2004. <https://ece.uwaterloo.ca/~z70wang/publications/ssim.pdf>
- [8] gsplat Documentation. Rasterization API. <https://docs.gsplat.studio/main/apis/rasterization.html>

- [9] Yu Z, Chen A, Huang B, Sattler T, Geiger A. Mip Splatting Alias Free 3D Gaussian Splatting. CVPR, 2024. <https://github.com/autonomousvision/mip-splatting>
- [10] Huang B, Yu Z, Chen A, Geiger A, Gao S. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. SIGGRAPH, 2024. <https://github.com/hbb1/2d-gaussian-splatting>
- [11] Chung J, Oh J, Lee K M. Depth Regularized Optimization for 3D Gaussian Splatting in Few Shot Images. CVPR Workshop, 2024. <https://github.com/robot0321/DepthRegularizedGS>
- [12] PlayCanvas. SuperSplat The 3D Gaussian Splat Platform. <https://dev.playcanvas.com/products/supersplat>